COSC-4117EL-01 — Artificial Intelligence, Fall 2026

Assignment 1: Futoshiki Puzzle Solver using Constraint Satisfaction

Group 19
Yapa Sugandi Upeksha - 0477736
Kankanala Ramya - 0477092

1. CSP formulation and application

An N × N puzzle has N^2 variables X(r,c), represented by one-based (row, column) tuples. Each variable initially has domain {1, …, N}; a given value v restricts its domain to {v}. Every pair of distinct cells sharing a row or column has a binary not-equal constraint. Each inequality clue adds a directed < or > constraint between adjacent cells. The enhanced solver stores both arc directions, reversing the inequality for the reverse arc. Supporting value pairs must satisfy all constraints between their cells. A solution assigns every variable while preserving the givens and satisfying every constraint.

The Python/Tkinter application loads section based TXT files, validates their contents, displays the grid and inequality clues, and offers both solvers. Fixed values appear in black and generated values in blue. Status, elapsed time, and assignment attempt counts are displayed after solving.

2. Algorithm design

Basic Backtracking recursively selects unfilled cells in row-major order and tries values 1 through N in ascending order. Before committing a value, it checks the applicable row, column, given-value, and inequality constraints whose endpoints are known. Failure causes backtracking. The baseline uses no propagation or search heuristics.

AC-3 Enhanced Backtracking initializes domains, removes given values from row/column peers, and runs AC-3 before search. REVISE(Xi, Xj) removes each value in Di lacking support in Dj. A changed Di causes incoming neighboring arcs to be reconsidered; an empty domain rejects the branch. Search selects a non-singleton domain using Minimum Remaining Values (MRV), breaking ties by the largest number of unresolved neighbors (degree). Least Constraining Value (LCV) tries values that eliminate the fewest neighboring values first. Remaining ties use row-major variable order and ascending values.

Each tentative assignment restricts its domain to a singleton and triggers AC-3 on incoming affected arcs. The solver copies the domain dictionary and every domain set before exploring alternatives, and restores the parent domains on failure. Therefore, pruning in one branch cannot corrupt another. Singleton domains need no search assignment. Complete solutions pass independent validation.

3. Experimental method

Both methods were tested on the same computer: Intel Core i7 at 3.00 GHz, approximately 16 GB RAM, Ubuntu 24.04.4 LTS, and Python 3.12.7. The default timeout was 60 seconds. Two additional runs used zero seconds to exercise timeout handling. Runtime uses *time.perf_counter()*, starting before solver initialization and including propagation, search, and final validation, but excluding file loading and GUI rendering. One node means one attempted value assignment, including rejected attempts. Givens and propagation-forced values do not count.

The CSV records 56 attempts across 35 files. 15 solvable puzzles tested with both methods, four unsolvable puzzles tested with both, 16 malformed loader tests, and two forced timeouts. **Easy puzzles retain 75% of givens and 35% of adjacent inequality edges, challenging puzzles retain 15% and 25%, respectively, rounded down**. **These are construction labels, not calibrated difficulty ratings.**

The separate validate_solution function checks exact cell coverage, integer ranges, row/column uniqueness, givens, and every inequality. All 30 solved records report True. Unsolvability is also supported independently. Three fixtures contain a strict inequality cycle, and the givens-only fixture forces a row value already present in its column. **A missing solution alone is not proof of unsatisfiability**.

4. Results and discussion

Appendix Tables A1 and A2 report every recorded result. Both methods solved all 15 solvable puzzles and rejected all four unsolvable puzzles within 60 seconds. All 16 malformed files were recorded as Invalid puzzle. Both zero-second attempts returned Time out, with small positive runtimes and zero attempted assignments.

On challenging 6×6 and 8×8 puzzles, **enhancement reduced nodes** from 92,345 to 12 and from 603,869 to 35, respectively. **Runtime fell** from 0.796936338 to 0.034756038 seconds (~23× faster) and from 5.973666553 to 0.127476678 seconds (~47× faster). AC-3 detected all four unsolvable cases without search assignments. **Propagation removes unsupported values early, while heuristics guide the remaining search**. These experiments do not isolate each component's contribution.

Lower node counts did not always mean lower runtime. On the 8×8 row-major-friendly puzzle, enhancement used 44 rather than 288 nodes but took 0.206395420 rather than 0.002254773 seconds. Constraint construction, repeated support checks, heuristic scans, and domain copying add work uncounted by the node metric. On the heuristic-detour puzzle, enhancement was worse on both measures, 392 versus 288 nodes and 1.220321798 versus 0.003020408 seconds. **Heuristics therefore provide no guarantee of a better search order**.

5. Limitations

Each configuration has one recorded measurement, so timing variability is unknown, especially for sub-millisecond runs. The small constructed dataset and uncalibrated difficulty labels limit generalization. Solution uniqueness is not established. Full domain copying and broad neighbor scans add overhead, and synchronous GUI solving can temporarily freeze the interface. Repeated measurements and separate heuristic comparisons would strengthen the evaluation.

6. Generative AI disclosure

ChatGPT and OpenAI Codex assisted with GUI development, validation helpers, solver integration and corrections, timing/node accounting, parser hardening, unit tests, sample puzzle generation, and this report optimization and corrections. Existing disclosures distinguish student algorithms from AI-assisted changes. Verification included the student's completed tests, independent solution validation, and regression tests with an exhaustive 3×3 oracle. This report was checked against the source code and recorded CSV. Reported timings were not replaced by new runs. Appendix A3 provides the recorded prompt links.

APPENDIX — COMPLETE RECORDED RESULTS AND SOURCES

Table A1. Solver results from test_results.csv (40 attempts).
Each row reports both methods on the same file and timeout.

BT = Basic Backtracking
AC = AC-3 Enhanced Backtracking.
N = Puzzle Grid Size

Times are seconds, and nodes are assignment attempts.

Status applies to both methods:

S = Solved
NS = No solution
TO = Time out.

Difficulty:
E = easy
C = challenging
U = not rated.

All S results have solution_valid=True, NS/TO results have solution_valid=N/A.

Table A1

ID	N	Diff	Limit	Status	BT time	BT nodes	AC time	AC nodes
01	4	U	60s	S	0.001030580	77	0.003810056	0
02	1	E	60s	S	0.000077219	1	0.000137395	0
03	4	C	60s	S	0.001242933	170	0.003913265	1
04	4	U	60s	S	0.000133018	0	0.001701643	0
05	4	E	60s	S	0.000482797	9	0.001717349	0
06	4	U	60s	S	0.000573861	40	0.010559203	8
07	4	U	60s	NS	0.001396776	172	0.001573405	0
08	4	U	60s	NS	0.000114853	4	0.000404312	0
09	6	C	60s	S	0.796936338	92345	0.034756038	12
10	6	E	60s	S	0.000496818	27	0.003203544	0
11	6	U	60s	NS	0.045378806	8772	0.002532900	0
12	8	C	60s	S	5.973666553	603869	0.127476678	35
13	8	E	60s	S	0.001227274	71	0.009528012	0
14	8	U	60s	NS	3.910316368	761000	0.021611716	0
15	4	E	0s	TO	0.000016732	0	0.000028667	0
16	4	E	60s	S	0.000144799	4	0.001049098	0
17	6	E	60s	S	0.001634636	126	0.014024728	0
18	6	E	60s	S	0.000461710	27	0.002895440	0
19	8	U	60s	S	0.002254773	288	0.206395420	44
20	8	U	60s	S	0.003020408	288	1.220321798	392

Table A1 file key (paths relative to the project directory):
01 puzzle_sample.txt
02 puzzles/valid/1x1_valid.txt
03 puzzles/valid/4x4_challenging.txt
04 puzzles/valid/4x4_complete.txt
05 puzzles/valid/4x4_easy.txt
06 puzzles/valid/4x4_empty.txt
07 puzzles/unsolvable/4x4_unsolvable.txt
08 puzzles/unsolvable/4x4_unsolvable_givens.txt
09 puzzles/valid/6x6_challenging.txt
10 puzzles/valid/6x6_easy.txt
11 puzzles/unsolvable/6x6_unsolvable.txt
12 puzzles/valid/8x8_challenging.txt
13 puzzles/valid/8x8_easy.txt
14 puzzles/unsolvable/8x8_unsolvable.txt
15 puzzles/valid/4x4_easy.txt (forced timeout)
16 puzzles/valid/4x4_single_blank.txt
17 puzzles/valid/6x6_inequalities_only.txt
18 puzzles/valid/6x6_reversed_endpoints.txt
19 puzzles/valid/8x8_row_major_friendly.txt
20 puzzles/valid/8x8_heuristic_detour.txt

A zero node count is a measured count, not a missing result. In AC-3, propagation can solve a puzzle or detect a contradiction before search. **The zero-second tests explicitly returned Time out**. Neither is recorded as a zero-second runtime or as No solution.

Table A2. Malformed input results (16 GUI loader attempts). All files below are in *puzzles/invalid/*. **Every recorded status of each of these files is Invalid puzzle**. Solver timeout, runtime, nodes, and solution validity are N/A because loading rejects the input before either solver runs.

Table A2
malformed_data_before_heading.txt
malformed_duplicate_column.txt
malformed_duplicate_row.txt
malformed_invalid_integer.txt
malformed_invalid_operator.txt
malformed_missing_heading.txt
malformed_nonadjacent.txt
malformed_out_of_order.txt
malformed_out_of_range.txt
malformed_repeated_cell.txt
malformed_repeated_givens.txt
malformed_repeated_inequalities.txt
malformed_repeated_size.txt
malformed_reversed_duplicate.txt
malformed_unknown_heading.txt
malformed_wrong_fields.txt

A3. Sources and AI prompts

https://chatgpt.com/s/cx_6ac9a7918b8881918cd92ac9eaac7337
https://chatgpt.com/s/cx_6ace5baac22c8191964a80d948bbdda9
https://chatgpt.com/s/cx_6ace8e68584c8191baee9a12783b575a

For this report, Codex inspected the assignment and local project files, checked CSV records and puzzle paths, verified all 16 malformed inputs were rejected by the parser, and calculated comparisons from the recorded measurements. Existing regression tests include 60 deterministic 3×3 cases checked against an exhaustive Latin-square oracle. The test suite was not rerun while writing this report.